

Da Coleta ao Dashboard: Roteiro do Pipeline

F105 – Datamining e Webscraping · Monitoramento de preços da cesta básica (São Luiz)

Fala em ordem cronológica, do jeito que o trabalho realmente aconteceu. Cada bloco abaixo é uma etapa do pipeline: o título resume o que aconteceu, e o texto em *itálico* é uma sugestão de como explicar aquela etapa com suas próprias palavras.

1. O problema

“A gente queria monitorar o preço de itens básicos — arroz, feijão, açúcar, óleo e ovos — no supermercado São Luiz, pra depois comparar marcas e ver qual é a mais barata e a mais cara de cada item.”

2. Por que aquele site foi um desafio

“Quando fomos olhar o código-fonte da página, vimos que ela é uma SPA em React — ou seja, o HTML que o servidor entrega vem vazio, tudo é montado depois pelo JavaScript no navegador. Isso significa que não dava pra simplesmente baixar a página e puxar os preços dali.”

3. Como achamos o caminho certo

“Em vez de usar uma ferramenta pesada tipo Selenium pra simular um navegador, fomos no DevTools do Chrome, na aba Network, e ficamos observando quais requisições o site fazia sozinho enquanto carregava os produtos. Foi aí que descobrimos que o site consome uma API própria — um endereço que devolve os dados já prontos em JSON: nome, preço, código de barras, estoque.”

4. O obstáculo da autenticação

“Essa API exige um token de autenticação, que o próprio navegador gera e que expira rápido — em poucos minutos. Isso deu trabalho: várias vezes o script parava de funcionar porque o token tinha vencido, e a gente tinha que pegar um novo token no DevTools e rodar de novo.”

5. A coleta em si (coleta.py)

“O script faz uma requisição por página, para cada uma das cinco categorias, com pausa entre elas pra não sobrecarregar o servidor. No final, juntamos 268 registros brutos.”

6. Um problema de categoria que descobrimos no meio do caminho

“Reparamos que arroz, feijão e açúcar têm categoria própria e limpa. Mas óleo e ovos, na verdade, vinham de categorias bem amplas do site — “Temperos” e “Mercearia” — que misturam vários outros produtos junto. Então tivemos que investigar a estrutura de categorias do próprio site pra entender isso.”

7. O tratamento dos dados (tratamento.py)

“Primeiro tiramos duplicados e registros sem nome ou preço. Depois padronizamos os preços num formato só. E, por causa daquele problema de categoria, criamos um filtro que só mantém produtos cujo nome realmente bate com o item que estamos buscando — por exemplo, só conta como “óleo” o que começa com “óleo de soja”, “óleo de milho”, etc., excluindo coisas como azeite ou óleo capilar que apareciam na mesma categoria. Isso derrubou os 268 registros pra 79 registros limpos e relevantes.”

8. A análise (analise.py)

“Com os dados limpos, calculamos estatísticas — média, mediana, mínimo, máximo — e geramos três gráficos: um de preço médio por item, um boxplot mostrando a variação de preço, e um comparando o mais barato com o mais caro de cada item. Depois juntamos os dois principais num dashboard único.”

9. A resposta à pergunta de negócio

“No fim, conseguimos responder: qual marca é mais barata e qual é mais cara em cada item, e qual a diferença percentual entre elas. O caso mais chamativo foi o arroz, com 783% de diferença — mas aqui vale explicar pro professor que isso compara um arroz comum com um arroz negro especial, produtos bem diferentes, não é só diferença de marca do mesmo tipo de produto.”

10. Uma limitação que a gente assumiu conscientemente

“O óleo ficou com só 4 registros depois do filtro, porque a categoria de origem era muito ampla. A gente decidiu seguir com esses 4 mesmo assim, em vez de arriscar uma nova coleta com token novo, e deixamos essa limitação documentada no relatório.”

11. Os cuidados éticos

“Os dados eram públicos, sem login. Não coletamos nenhuma informação pessoal. E, como a API não é documentada oficialmente, tratamos isso com cuidado: pausas entre requisições, coleta limitada, token guardado como variável de ambiente e não no código.”